Max Capital — Challenge Técnico

Senior Software Engineer — Equipo de Arquitectura (Backend)

Contexto

Max Capital recibe, en tiempo real, los reportes de ejecución (*ExecutionReports*) que el mercado emite sobre las órdenes operadas. Queremos ver cómo diseñás y construís un servicio que consuma ese flujo de forma **asíncrona**, mantenga el estado de cada orden **correcto** bajo carga y fallas, corriendo en **dos instancias** en paralelo.

No buscamos un sistema completo ni con muchas features. Buscamos que **el estado de cada orden sea siempre correcto** —incluso ante reentregas y fallas—, y que tus **decisiones y trade-offs estén explícitos**. Un servicio chico con las cosas difíciles bien resueltas vale mucho más que un sistema grande con features que no pedimos.

No necesitás experiencia en mercados: todo lo que hay que saber está más abajo.

> **Terminología** — Para evitar confusiones: **orden** se refiere siempre a la *orden de mercado* (identificada por `numericOrderId`); **secuencia** se refiere al *orden temporal* en que se emiten y aplican los ER.

El mensaje

Cada evento es un *ExecutionReport* (ER): una foto del estado de una orden en un instante. La estructura real es parecida a esto:

```
ExecutionReport(
  fixId=523130930000307,
  numericOrderId=13144742,            // identifica la ORDEN (se repite en todos sus ER)
  marketOrderId="O0S6tDQoQqVy",
  ticker="VSCPC",
  side=BUY,
  securityType=COMMON_STOCK,
  status=PARTIALLY_FILLED,            // NEW | PARTIALLY_FILLED | FILLED | CANCELLED
  orderPrice=104.25,
  nominalAmounts=4956,                // cantidad total de la orden
  leavesNominalAmount=1200,           // cantidad que resta operar
  accumulativeNominalAmount=3756,     // cantidad acumulada operada hasta este ER
  executionNominalAmount=800,         // cantidad de ESTA ejecución
  executionPrice=104.30,
  avgPrice=104.28,
  secondaryTradeId="...",             // identidad de la ejecución (para dedup)
  operationNumber="...",              // idem
  transactionTime=2026-07-20T18:08:52.129,
  ...
)
```

Lo que importa entender del dominio:

- Una **orden** (`numericOrderId`) genera **muchos ER a lo largo de su vida**: **siempre arranca con un `NEW`**, luego vienen los `PARTIALLY_FILLED`, y termina en `FILLED` o `CANCELLED`. Todos comparten `numericOrderId`.
- Los campos de cantidad son **acumulados (snapshots), no deltas**: cada ER trae el acumulado total operado (`accumulativeNominalAmount`) y lo que resta (`leavesNominalAmount`).
- `transactionTime` es un reloj de pared del origen: **no es confiable como criterio de secuencia**.

El flujo y la garantía

Vas a recibir un **stream multiplexado**: llegan ER de muchas órdenes distintas, intercalados entre sí. Es decir, entre dos ER de la orden `A` pueden aparecer ER de las órdenes `B`, `C`, etc. Podés asumir que el broker entrega los ER en el orden en que fueron emitidos.

La garantía que tu servicio debe cumplir:

> Los ER de una **misma orden** deben aplicarse sobre el estado de esa orden en la secuencia en que fueron emitidos (p. ej., no se puede aplicar un `FILLED` antes del `NEW` de esa orden). En cambio, entre **órdenes distintas** no importa el orden relativo: pueden procesarse en paralelo.

El servicio corre en **dos instancias** consumiendo el mismo flujo. Cómo lográs esa garantía de secuencia por orden con dos consumidores en paralelo lo decidís vos.

Qué tenés que construir

1. **Ingesta asíncrona** de ER a través de un message broker. La elección del broker y del mecanismo es tuya.
2. **Dos instancias** del servicio corriendo en paralelo (docker-compose), ambas consumiendo.
3. **Secuencia de aplicación garantizada por orden**, con paralelismo entre órdenes distintas, según la garantía de arriba.
4. **Idempotencia:** un ER duplicado o reentregado no debe corromper el estado. (Atención: *muchos ER por orden es lo normal*, no un duplicado — la identidad de un duplicado es el ER individual, no la orden.)
5. **Entidad orden con estado mutable, persistente y consultable.** Además de un **ledger de ejecuciones** (una entrada por ER **efectivamente aplicado**, con una **key autoincremental** que refleje el orden de inserción; un duplicado detectado **no** agrega entrada), mantenés una **entidad orden** que se actualiza a medida que llegan sus ER. Actualizarla **no es sobreescribir con el último ER**: el nuevo estado se **computa a partir del estado ya persistido más el ER entrante**. Como mínimo debe incluir:
6. el **`status` actual**, cuya transición se evalúa contra el `status` ya guardado (un ER no puede aplicarse sobre una orden ya terminal, `FILLED` / `CANCELLED`);
7. la **cantidad de ejecuciones aplicadas**, que se incrementa sobre el valor anterior.

El estado de la orden debe reflejar **exactamente** la secuencia de ER que aplicó. Exponé un **endpoint HTTP para consultar una orden por `numericOrderId`** que devuelva su estado actual (`status`, cantidad de ejecuciones aplicadas) junto con el detalle de su ledger en orden de inserción. El modelo de persistencia lo elegís vos. 6. **Recuperación ante fallas:** si una instancia se cae a mitad de procesamiento, no se pierden ni se aplican dos veces los ER; al reiniciar, retoma correctamente. 7. **Manejo de errores de procesamiento.** Un ER que falla al procesarse no puede provocar **pérdida silenciosa** (una orden a la que le falta un ER sin que nadie se entere) ni **bloqueo indefinido** del flujo de esa orden. Distinguí errores **transitorios** (reintentables) de **permanentes** (mensaje inválido / envenenado), y contemplá qué pasa con una orden que queda incompleta. No es obligatorio implementar todo el mecanismo, pero el descarte silencioso no es una opción. 8. **Efecto terminal:** cuando una orden se completa (`status = FILLED`), el servicio debe **publicar un mensaje de liquidación** (`settlement`) hacia un destino downstream (otra cola/topic), de modo que downstream lo reciba **exactamente una vez** por orden — sin duplicados ni pérdidas —, aun cuando haya reentregas de ER o las dos instancias vean el completado (típicamente vía idempotencia / outbox / dedup). **No hace falta implementar el consumidor** de ese `settlement`: basta con publicarlo. Las órdenes que terminan en `CANCELLED` **no** emiten settlement.

Qué evaluamos

- **Correctitud de las garantías** de arriba, específicamente con **2 instancias + reentrega**.
- **Claridad de los trade-offs**: qué elegiste, por qué, y qué resignaste conscientemente.
- **Pragmatismo**: priorizar el núcleo por sobre lo accesorio.
- **Criterio de testing**: identificar los puntos más frágiles del sistema y escribir pruebas significativas sobre ellos, en lugar de acumular volumen superficial.

No puntuamos: UI linda, features extra, hardening de producción.

No hay una única solución correcta: evaluamos el razonamiento detrás de tus decisiones tanto como el resultado.

Entregables

1. **Repo** con `docker compose up` que levante todo: **2 instancias** del servicio + broker + persistencia + lo que hayas decidido sumar.
2. Una forma de **emitir un stream de ER de prueba** (script, endpoint o seed) con varias órdenes intercaladas y algún ER duplicado, más una nota breve en el README de **cómo ejercitar los escenarios clave** (órdenes intercaladas, duplicados, caída de una instancia) para que podamos reproducirlos por nuestra cuenta.
3. **`DECISIONS.md` — el porqué de tus decisiones.** Es donde justificás el diseño; lo leemos junto con el código y es el punto de partida de la entrevista. Cubrí al menos:
4. elección de broker y por qué,
5. estrategia para garantizar la secuencia por orden con 2 consumidores, y qué resignás,
6. estrategia de idempotencia (y cuál es tu clave de dedup),
7. cómo persistís el estado y por qué elegiste ese motor,
8. cómo garantizás que el estado de la entidad orden refleje exactamente su secuencia de ER,
9. política ante errores de procesamiento: reintentos, dead-lettering, y cómo evitás perder o duplicar un ER en el proceso,
10. cómo garantizás que el `settlement` se emita una sola vez (sin duplicados ni pérdidas),
11. trade-offs que dejaste afuera a propósito y cómo los resolverías en una versión completa.
12. **Tests focalizados.** No buscamos cobertura amplia ni tests de código trivial. Incluí tests sobre lo que consideres crítico para validar las partes más complejas del núcleo.
13. Instrucciones breves de cómo correrlo.

Restricciones

- **Java 21 o 25** + **Spring Boot** para el servicio.
- Enfocate en el **núcleo** (las garantías). Si algo queda afuera, alcanza con dejarlo documentado en `DECISIONS.md` en vez de implementarlo; no hace falta que sea production-ready.
- No construyas: auth, UI real, features extra, ni hardening de producción.
- Cualquier duda de alcance, resolvela vos y dejá la suposición anotada en `DECISIONS.md`.

Entrega

Entregá el **código fuente completo en un repositorio git público** (link a GitHub/GitLab), junto con las instrucciones para correrlo con `docker compose up`.

El código es el entregable central: lo vamos a leer y ejecutar, y la entrevista técnica posterior parte de tu solución y tu `DECISIONS.md`. Conservá tu historial de commits.